magic-haskell

Haskell listings that link to Hackage and talk to GHC

Florian Sihler*

Version v1.0 – 2026/09/29

github.com/EagleoutIce/magic-haskell

Contents

1 Introduction 1
2 Automatically Linking Names 3
3 Typesetting Code 5
4 Computing Answers with GHC 6
5 Known Limitations 8

1 Introduction

`magic-haskell` extends the `xlistings` package with a Haskell language that automatically links "magic" names to their documentation on Hackage or to their definition in the document. It can also ask GHC for types, values, and whole sessions. Originally, I developed `magic-haskell` privately for the functional programming lecture at Ulm University.

```haskell
data Shape = Circle Double | Square Double
type AnotherCirc = Circle

area :: Shape -> Double
area (Circle r) = pi * r * r
area (Square a) = a * a

main :: IO ()
main = mapM_ (print . area) [Circle 1, AnotherCirc 2] -- no links in <$> comments
```

Every linked name is underlined in gray. `Double` and `mapM_` lead to their documentation, `Shape`, `Circle`, and `area` to their definitions above. Names are matched as whole words, and never within strings or comments (however, the highlighting is not fully context sensitive).

1.1 Features at a Glance

There are many features that `magic-haskell` adds on top of `xlistings` and it interacts flawlessly with its styles, line numbers, environments, inline commands, and `minted` emulation layer, as well as with the optional `code-animation` package (Section 3.3). All of the following happens automatically for Haskell code:

- **Links to Hackage.** Known names and operators link to their documentation, e.g. `mapM_ print xs`. By default, this version of `magic-haskell` ships with 7 217 links for 262 modules that were extracted, verified, and cached automatically from a set of desired packages. They are stored in its link database and can be extended easily (Sections 2.1 and 2.2).

*Released under the LaTeX Project Public License, version 1.3c or later.

- **Links and syntax highlighting for user definitions.** Functions, types, and constructors are recognized where they are defined, and every use links there, anywhere in the document and across beamer overlays: `area (Circle 2)` leads back to the listing above. Local `where` and `let` bindings stay defined only within their listing (Section 2.4).
- **Adaptive Redefinitions.** after a second `data Foo`, later uses link to it, while earlier ones keep linking to the first. On beamer slides, links lead to an overlay that shows the definition in full (Section 2.4).
- **Qualified names.** `Data.Map.insert` links sensitive to the import. This also works with aliases, e.g., `M.insert` (not active here) if an import of the listing names the alias `M` (Section 2.5).
- **Ligatures copy as text.** `f <$> xs >>= g` can be typeset with ligatures, but copies as `f <$> xs >>= g` if accessibility support is enabled and supported by the PDF viewer (Section 3.1).
- **Markers.** Support for multi-line highlight segments with `head xs`, `undefined`, and more (Section 3.2).
- **Answers from GHC.** We can request GHC to compute types and values during document compilation, e.g. `\mhsType{map}` returns `(a -> b) -> [a] -> [b]` (Section 4).
- **GHCi sessions.** The environment `ghcisession` runs its lines in GHCi and typesets the answers. On beamer slides, they appear one overlay per step, or all at once with `animate=false` (Section 4.2).

1.2 Loading the Package

The `magic-haskell` package consists of three files. The main package in `magic-haskell.sty`, the GHC adapter in `magic-haskell-ghc.sty`, and the link database in `magic-haskell-links.cfg`. We require `xlistings` to work (hooking into its existing processor layer), which the repository embeds as a git submodule (clone with `--recurse-submodules`). Links require `hyperref`, copyable ligatures `accsupp`.

```
\usepackage{magic-haskell}        % optionally with [ghc] for the GHC integration
\usepackage{hyperref}
```

The package registers two languages, each with the environments and commands of `xlistings`:

- `mhs` (configurable via the option `language`, alias `magic-haskell`) with the environments `mhs`, `mhs*` (no line numbers), `plainmhs` (no styling and no line numbers), and the inline commands `\bmhs`, `\cmhs`, and `\imhs` (input a file), e.g. `foldr (+) 0 <$> Just [1, 2, 3]`. This document uses `language=hs`, hence `\bhs`.
- `ghci`, a session where the prompt `ghci>` is not copyable and `out>` marks output to have no highlighting:

  ```
  ghci> :t fmap
  fmap :: Functor f => (a -> b) -> f a -> f b
  ```

  Please note that this language just defines the markup, see Section 4.2 for the environment `ghcisession` that runs its lines automatically with GHCi.

With the `fakeminted` option of `xlistings` (on by default), this also works with the *minted* environment.

1.3 Package Options

- **`links`** (default `true`) links known names. `\mhsLinksOn` and `\mhsLinksOff` switch it within the document:

  ```
  \bhs{map show xs}                % with links
  {\mhsLinksOff \bhs{map show xs}}  % without
  ```

  give `map show xs` and `map show xs`.

- **`underline`** (default `true`) underlines linked names in the color `mhslink`. `\mhsUnderlineOn` and `\mhsUnderlineOff` toggle this behavior:

  ```
  {\mhsUnderlineOff \bhs{map show xs}}
  {\colorlet{mhslink}{cyan!40}\bhs{map show xs}}
  ```

  give `map show xs` and `map show xs`.

- **`ligatures`** (default `true`) typesets operators as ligatures (Section 3.1). After `\mhsLigaturesOff`, `f <$> xs` will be rendered as `f <$> xs`; `\mhsLigaturesOn` switches ligatures on again.

- **`tight`** (default `true`) moves the characters of operators closer together. `\mhsTightOn` and `\mhsTightOff` switch it, so `f <$> xs >>= g` becomes `f <$> xs >>= g`.

- **packages** (default `all`) selects the packages of the database to link (this is useful if many packages export the same names and you want to specify which of those are to be considered loaded):

  ```
  \usepackage[packages={base,mtl}]{magic-haskell}
  ```

- **database** (default `magic-haskell-links`) names the link database, a `.cfg` file, e.g. one generated from a package list of your own (Section 2.2):

  ```
  \usepackage[database=my-links]{magic-haskell} % reads my-links.cfg
  ```

- **language** (default `mhs`) names the language, its environments, and its commands. This document uses `language=hs`, so its code is written as `\begin{hs}` and `\bhs{...}`. As a small support for editors with latex-sensitive syntax highlighting, we support `\$` in the argument of `\bhs`, e.g. `\bhs{f <\$> xs}` (`f <$> xs`), so that editors do not take it for a TEX-math environment.
- **ghc** (default `false`) loads the GHC integration presented in Section 4.
- **xlistings** (no default) can be used to pass options to `xlistings`, for example to change the default style:

  ```
  \usepackage[xlistings={style={plain number},highlights}]{magic-haskell}
  ```

2 Automatically Linking Names

2.1 The Link Database

In this version, the database `magic-haskell-links.cfg` holds 7 217 links into 262 modules of all packages in Table 1. It is generated by the bash script `tools/generate-links.sh` (on Windows, run it in Git Bash or use the WSL) from the list `tools/packages.txt`:

```
# package version (or 'latest') modules ('*': all remaining ones)
base 4.22.0.0 Prelude Data.List Data.Maybe ...
mtl 2.3.2 Control.Monad.State.Class ...
base 4.22.0.0 *
```

A name exported by several modules links to the first of them, so `map` links to the `Prelude` and not to `Data.List`. Section 2.3 shows how to choose and change the module load order or the modules that are considered to be loaded at all. A package may appear more than once, e.g. to rank the rarely used modules of `base` last. The version `latest` takes the newest version with documentation on Hackage.

Table 1: The packages of the link database, as loaded by this document.

Package	Version	Modules	Links
base	4.22.0.0	217	5 761
transformers	0.6.3.0	6	84
mtl	2.3.2	4	29
containers	0.8	2	199
array	0.5.8.0	2	15
bytestring	0.12.2.0	1	117
text	2.1.4	1	109
random	1.3.1	1	63
parsec	3.1.18.0	25	517
QuickCheck	2.19.0.0	1	259
hspec	2.11.17	1	43
hspec-expectations	0.8.4	1	21
Total		262	7 217

The names are read from the anchors of the module pages. Afterwards, the script recomputes every link as `magic-haskell` does and checks that its page and anchor exist. `tools/generate-links.sh --verify` repeats the check at any time to ensure that the database is still valid.

2.2 Extending the Link Database

To link another package or more modules, add them to `tools/packages.txt` and regenerate the database. The script needs `bash`, `curl`, and `jq`:

```
# in tools/packages.txt: package, version ('latest' for the newest), modules
vector latest Data.Vector Data.Vector.Mutable
# regenerate magic-haskell-links.cfg and write the versions used into the list
tools/generate-links.sh --pin
```

You can also use and create a database of your own:

```
tools/generate-links.sh -o my-links.cfg my-packages.txt
```

You can then load it with the option `database=my-links`:

```
\usepackage[database=my-links]{magic-haskell}
```

Single names or links to pages outside of Hackage, can be added in the document itself (Section 2.6).

2.3 Choosing Modules for Shared Names

`\mhsModules` restricts the links to the given modules and ranks them in the given order. The key `modules` does so for a single listing, and an empty list activates all modules again:

```
\mhsModules{Data.Map.Strict,Prelude} % from now on
\begin{hs}[modules={Data.Set}]    % in this listing
```

With `\mhsModules{Data.Map.Strict,Prelude}`, `insert k v m` links `insert` to `Data.Map.Strict`, while it links to `Data.List` by default. Names that no active module exports are neither linked nor highlighted.

2.4 Linking to Definitions in the Document

A displayed listing defines the names bound in its first column (`area :: ...` or `area x = ...`) and the types after `data`, `newtype`, `type`, and `class`, and the constructors of a `data` or `newtype`, which keep the plain color and are only underlined. Every use of such a name links to its definition: in the same listing if it defines the name, else the last definition before, else the first one after. On beamer slides, a link from outside a frame goes to the last overlay of that frame, where all of its code is visible, and code that an overlay hides (`\only`, `onlyenv`) defines nothing there. Definitions shadow the names of the database, so the introduction's `area` does not link to any library. The definitions are kept in the `.aux` file, hence they need a second run.

The bindings of a `where` or `let` block are local definitions, which only count within their listing. As in Haskell's layout rule, every line that starts in the column of the first binding binds another name, so `total`, `count`, and `step` below all link within the listing, while `foldr` links to Hackage:

```
average :: [Double] -> Double
average xs = total / count
  where
    total = foldr (+) 0 xs
    count = foldr step 0 xs
    step _ n = n + 1
```

Inline code and GHCi sessions are never assumed to define something. The key `mhsdefinitions` of a listing controls its definitions: `false` disables the definition tracking, and `local` keeps all of them within the listing, e.g. for a self-contained example that should not affect the links of other listings:

```
\begin{hs}[mhsdefinitions=local]
```

2.5 Resolving Qualified Names and Aliases

Qualified names link into their module: in `Data.Map.insert k v m`, `insert` links to `Data.Map.Strict`, the first module of the database whose name starts with the qualifier. Aliases come from the imports of the listings, so after `import qualified Data.Map.Strict as M`, `M.lookup` links there, too. A name in lower case before the dot is considered to be a composition: in `show.map`, both names link on their own.

2.6 Adding Links in the Document

- `\mhsLinks[package]{Module}{names}` links names into `Module` of the Hackage `package` (default `base`). Names in upper case are types, other names are values. You have to escape `$` and `&`.
- `\mhsConstructors[package]{Module}{names}` links data constructors, styled like types but linked as values.
- `\mhsAddLink[style]{url}{name}` links a name to any url. The style is `type`, `func`, `op`, or `plain` (default), and the link wins over the database: `\mhsAddLink[func]{https://example.org/frob}{frob}`.
- `\mhsNoLink{names}` marks names that are neither to be linked nor highlighted (as a kind of blacklist).
- `\mhsSetVersion{package}{version}` the version in the links of a package. An empty one links to the latest release. It applies to all links typeset after it.

3 Typesetting Code

3.1 Ligatures That Copy as Text

Operators become ligatures whose copy is the original text (with accsupp). Tight ligatures move their characters closer together, like programming fonts do, but keep their width, so columns still align. These are, of course, opinionated and depend on the used font as well:

<$> <$ $> <*> <* *> <&> <|> >>= =<< >=> <=< <> ++ !! &&
|| /= <= >= ==> $!

With `\mhsTightLigature{operator}` you can add your own ligature and `\mhsLigatureKern` (default `.12em`) is the distance the characters move so the library may compensate offsets. `\mhsLigature{operator}{code}` typesets an operator with arbitrary TEXcode, e.g. to use a different font, symbol, or a cute penguin:

```
\mhsLigature{<~}{\ensuremath{\leftsquigarrow}}
```

3.2 Escaping to LaTeX and Marking Segments

Code between `!*` and `*!` is interpreted as LaTeX, e.g. to annotate a line:

```
\begin{hs}
fib :: Int -> Int -- !*\enquote{naive}, see \emph{memoization}*!
\end{hs}
```

This produces:

```
fib :: Int -> Int -- “naive”, see memoization
```

Segments are marked with `|warn|...|warn|`, `|err|...|err|`, and `|info|...|info|`. They get a soft band behind the segment, which keeps its colors and links. These may span several lines. `|num|...|num|` marks a number, `|dim|...|dim|` fades a segment, and `|plain|...|plain|` turns off all highlighting. Markers are disabled within comments:

```
\begin{hs}
head [] = |err|undefined|err|
first  = |warn|head|warn| xs
safe xs = |info|listToMaybe|info| xs
answer = |num|forty-two|num| |dim|(or 42)|dim|
|plain|map|plain| f xs
\end{hs}
```

```
head [] = undefined
first   = head xs
safe xs = listToMaybe xs
answer  = forty-two (or 42)
map f xs
```

A marker may span several lines. Its bands intentionally cover only the code but not the indentation of a line:

```
\begin{hs}
main = do
  |warn|let ys = map (+1) xs
      zs = filter even ys|warn|
  print zs
\end{hs}
```

```
main = do
  let ys = map (+1) xs
      zs = filter even ys
  print zs
```

3.3 Animating Code Step by Step

`magic-haskell` works with, but does not need, `code-animation`, which walks through a listing on beamer slides: it fades out all lines but the current one and adds comments. Load it next to `magic-haskell` and wrap any listing of the language:

```
\usepackage{code-animation}
...
\begin{frame}[fragile]
\AnimateCode{onslide={1,2,/3: the recursion,0}}
\begin{hs}
size :: Tree -> Int
size Leaf = 1
size (Node l r) = size l + size r
\end{hs}
\endAnimateCode
\end{frame}
```

4 Computing Answers with GHC

With the option `ghc` (or `\usepackage{magic-haskell-ghc}`) we load an adapter that allows interfacing with GHCi. All interactions are cached and only re-run if the code changes.

4.1 Types and Values of Expressions

- `\mhsType{expr}` returns the inferred type of an expression, with a verbatim argument. Hence, the expressions

  ```
  \mhsType{map}            % just prints the type
  \mhsType*{foldr (:) []}  % with the expression
  ```

  produce `(a -> b) -> [a] -> [b]` and `foldr (:) [] :: Foldable t => t a -> [a]`.
- `\mhsEval{expr}` evaluates an expression. `\mhsEval{take 5 (iterate (*2) 1)}` produces `[1,2,4,8,16]`.
- `\mhsTypeOf`, `\mhsEvalOf` work the same but without taking the argument verbatim , e.g. in an escape `-- !*\mhsTypeOf{fib}*!` of a listing.
- `\mhsGhcTypeInto\macro{expr}`, `\mhsGhcEvalInto\macro{expr}` store the type or the value in `\macro`.
- `\mhsGhcIfError{true}{false}` indicates whether the last query failed. Failures show **??** and write the diagnostic of GHC to the log, missing answers show as **??**.

- `\mhsGhcTime`, `\mhsGhcMemory` the time GHCi took for the last evaluation and the memory it allocated, from its statistics (`:set +s`), always as plain numbers: the time in milliseconds and the memory in bytes. They are expandable, e.g. for computations. `\mhsGhcTimeText` and `\mhsGhcMemoryText` format them. For example, `\mhsEval{sum [1 .. 10^6]}` gives `500000500000` in 30 ms, allocating 88,108,136 bytes (the raw values are `30` and `88108136`). Commands like `:t` are no evaluations and keep the values. Please note that, because GHCi also has to interpret expressions, these measurements might be only a rough estimate.

4.2 Typesetting GHCi Sessions

The environment `ghcisession` runs its lines, one after the other, in a fresh GHCi:

```
\begin{ghcisession}
let xs = [1 .. 5]
sum xs
:t foldr
\end{ghcisession}
```

```
ghci> let xs = [1 .. 5]
ghci> sum xs
15
ghci> :t foldr
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
```

On beamer slides, the steps appear one per overlay by default. Steps that are yet to come take their space invisibly, so the slide does not move while the session grows. The options of a session are `animate` (to control whether the steps appear one per overlay), `start` (the overlay to start the animations with), and `listing` (options for the listing):

```
\begin{ghcisession}[start=2,listing={numbers=left}]
```

`\mhsGhcSessionSetup{options}` sets these options for all following sessions.

4.3 Writing and Loading Haskell Files

The environment `mhsfile[listing options]{file.hs}` writes its content to `file.hs`, typesets it, and answers following queries in the scope of that file:

```
1 fib :: Integer -> Integer
2 fib n = go n 0 1
3   where go 0 a _ = a
4         go k a b = go (k - 1) b (a + b)
```

Now `\mhsType{fib}` is `Integer -> Integer` and `\mhsEval{map fib [1 .. 10]}` gives

```
[1,1,2,3,5,8,13,21,34,55]
```

`\mhsGhcLoad{file.hs}` switches to an existing file, `\mhsGhcUnload` back to the `Prelude`.

4.4 How We Interact with Ghc

All queries of a run are written to the GHCi script `\jobname.mhs.ghci`, each after a marker line. A single GHCi session answers all questions by working with `\jobname.mhs.out`, which is read by the next run. GHCi only runs if the script changed (its md5 sum is kept in the `.aux` file). GHCi runs automatically if shell escape is unrestricted (`-shell-escape`). Otherwise the log contains the command to run by hand. The command is written for `sh`, and on Windows for `cmd.exe`.

- `\mhsGhcCommand{cmd}` the GHC executable. By default, the first of `ghc` on the `PATH`, the `ghc` of `ghcup`, and `stack exec -- ghc`; on Windows, the `ghc` on the `PATH`. You can use this, for example, to guard against endless evaluations using: `\mhsGhcCommand{timeout 60 ghc}`.
- `\mhsGhcFlags{flags}` can be used to supply further flags, e.g. `-XGADTs` or `-package containers`.

- `\mhsGhcRun{mode}` `auto` (default), `always`, or `never` to run GHCi automatically, or to just write the script and log the command to run by hand.
- `\mhsGhcSetup{line}` a line to execute at the start of the script, e.g. `:set -XOverloadedStrings`.
- `\mhsGhcImport{modules}` modules imported in every scope, e.g. `Data.List,Data.Char`.
- `\mhsGhcSetFormat{code}` configures how answers are shown, using `#1` as the answer (default: like `\bmhs`). `\mhsGhcSetErrorFormat` does the same for errors.

5 Known Limitations

- Operators of one character (except `$`) are never linked as this would break the highlighting of numbers like `.5` and of comments.
- Names of the document are only found in listings that are typeset with this package. Furthermore this linkage requires a second run.
- Local bindings are found by their indentation only. Layout with explicit braces and semicolons is not supported. We also only find the first name in pattern binding like `(a, b) = ...`.
- Operators right after a letter cannot be linked (`x<$>y` vs. `x <$> y`) due to parsing ambiguities with special infix operators.

Index of Commands, Options, and Keys

`animate` (key), 7

`\bmhs`, 2

`\cmhs`, 2

`database` (package option), 3

`ghc` (package option), 3
`ghcisession` (environment), 2, 7

`\imhs`, 2

`language` (package option), 3
`ligatures` (package option), 2
`links` (package option), 2
`listing` (key), 7

`mhs` (environment), 2
`mhs*` (environment), 2
`\mhsAddLink`, 5
`\mhsConstructors`, 5
`mhsdefinitions` (key), 4
`\mhsEval`, 6
`\mhsEvalOf`, 6
`mhsfile` (environment), 7
`\mhsGhcCommand`, 7
`\mhsGhcEvalInto`, 6
`\mhsGhcFlags`, 7
`\mhsGhcIfError`, 6
`\mhsGhcLoad`, 7
`\mhsGhcMemory`, 7
`\mhsGhcMemoryText`, 7
`\mhsGhcSessionSetup`, 7
`\mhsGhcTime`, 7
`\mhsGhcTimeText`, 7
`\mhsGhcTypeInto`, 6
`\mhsGhcUnload`, 7
`\mhsLigature`, 5
`\mhsLigatureKern`, 5
`\mhsLigaturesOff`, 2
`\mhsLigaturesOn`, 2
`\mhsLinks`, 5
`\mhsLinksOff`, 2
`\mhsLinksOn`, 2
`\mhsModules`, 4
`\mhsNoLink`, 5
`\mhsSetVersion`, 5
`\mhsTightLigature`, 5
`\mhsTightOff`, 2
`\mhsTightOn`, 2
`\mhsType`, 6
`\mhsTypeOf`, 6
`\mhsUnderlineOff`, 2
`\mhsUnderlineOn`, 2
`modules` (key), 4

`packages` (package option), 3
`plainmhs` (environment), 2

`start` (key), 7

`tight` (package option), 2

`underline` (package option), 2

`xlistings` (package option), 3